

Resolución: Conteo Eficiente de Inversiones en una Lista

Este documento presenta la resolución detallada del problema de calcular el número de inversiones en una lista $L[1 \dots n]$ utilizando la estrategia de **Divide y Vencerás** mediante una adaptación directa del algoritmo de ordenación **MergeSort**.

a) Máximo Número de Inversiones en una Lista de Longitud n

Respuesta

El máximo número de inversiones que podemos encontrar en una lista de longitud n es:

$$\frac{n(n-1)}{2} = \binom{n}{2}$$

Justificación

Una inversión se define como un par de índices (i, j) tales que $i < j$ y $L[i] > L[j]$.

1. **Número total de pares:** En cualquier lista de longitud n , el número total de pares de índices posibles (i, j) con $i < j$ corresponde a las combinaciones de elegir 2 elementos de entre un conjunto de n , lo cual equivale a:

$$\binom{n}{2} = \frac{n!}{2!(n-2)!} = \frac{n(n-1)}{2}$$

2. **Caso de máxima inversión:** El número máximo de inversiones ocurre cuando **todos** los pares posibles de la lista forman una inversión. Esto sucede si y solo si la lista está ordenada de manera **estrictamente decreciente** (por ejemplo, $L = [n, n-1, \dots, 2, 1]$). En este escenario, para cualquier par de índices $i < j$, siempre se cumplirá que $L[i] > L[j]$.

Dado que no pueden existir más parejas de índices que las $\frac{n(n-1)}{2}$ totales, este valor representa el límite superior absoluto de inversiones posibles.

b) Descripción del Algoritmo Eficiente (Basado en MergeSort)

Para diseñar un algoritmo de coste $O(n \log n)$, adaptamos el funcionamiento de **MergeSort** (Ordenación por Mezcla). La gran ventaja de este enfoque es que, al fusionar dos sublistas ya ordenadas, podemos detectar y contabilizar múltiples inversiones de golpe (en bloque) sin necesidad de comparar cada elemento de uno en uno.

Idea de la Adaptación

1. **Dividir (Divide):** Dividimos la lista $L[izq \dots der]$ por la mitad calculando el punto medio mid .

2. **Conquistar (Conquer):** De forma recursiva, ordenamos y contamos las inversiones de la mitad izquierda $L[izq \dots mid]$ y de la mitad derecha $L[mid + 1 \dots der]$.
3. **Combinar (Merge):** Fusionamos (mezclamos) las dos mitades ya ordenadas en una estructura temporal y, durante este proceso, contamos las **inversiones cruzadas** (elementos de la mitad izquierda que son estrictamente mayores que elementos de la mitad derecha).

Lógica de Conteo en el Proceso de Mezcla (Merge)

Supongamos que estamos mezclando la mitad izquierda $L[izq \dots mid]$ y la mitad derecha $L[mid + 1 \dots der]$, ambas ya ordenadas. Usamos dos punteros para recorrerlas: i para la izquierda y j para la derecha.

- Si $L[i] \leq L[j]$: El elemento $L[i]$ se coloca en su posición correcta en el vector temporal. No hay inversión. Avanzamos el puntero i .
- Si $L[i] > L[j]$: Se ha detectado una inversión. Como la mitad izquierda ya está ordenada ascendentemente, **todos** los elementos que queden en la mitad izquierda (desde la posición actual i hasta mid) son garantizadamente mayores que $L[j]$.
 - Por tanto, el elemento $L[j]$ forma exactamente $(mid - i + 1)$ inversiones con la sublista izquierda.
 - Sumamos este valor directamente al contador de inversiones y avanzamos el puntero j .

c) Algoritmo en Pseudocódigo

A continuación se detalla el pseudocódigo del algoritmo estructurado con índices base 1:

```

Funcion MergeYContar(L, izq, mid, der)
Inicio
  temp <- Vector de tamaño (der - izq + 1)
  i <- izq      // Puntero para la mitad izquierda
  j <- mid + 1  // Puntero para la mitad derecha
  k <- 1        // Puntero para el vector temporal
  inversiones_cruzadas <- 0

  Mientras i <= mid y j <= der Hacer
    Si L[i] <= L[j] Entonces
      temp[k] <- L[i]
      i <- i + 1
    Sino
      temp[k] <- L[j]
      // Sumamos todas las inversiones que produce L[j] con el bloque izqui
      inversiones_cruzadas <- inversiones_cruzadas + (mid - i + 1)
      j <- j + 1
    FinSi
    k <- k + 1
  FinMientras

  // Copiar elementos restantes de la mitad izquierda (si quedan)
  Mientras i <= mid Hacer
    temp[k] <- L[i]

```

```

        i <- i + 1
        k <- k + 1
FinMientras

// Copiar elementos restantes de la mitad derecha (si quedan)
Mientras j <= der Hacer
    temp[k] <- L[j]
    j <- j + 1
    k <- k + 1
FinMientras

// Volcar el contenido del arreglo temporal de vuelta a la lista original L
Para p <- 0 Hasta (der - izq) Con Paso 1 Hacer
    L[izq + p] <- temp[p + 1]
FinPara

Retornar inversiones_cruzadas
FinFuncion

Funcion MergeSortInversiones(L, izq, der)
Inicio
    Si izq >= der Entonces
        Retornar 0 // Caso base: un solo elemento no tiene inversiones
    FinSi

    mid <- ParteEntera((izq + der) / 2)

    // Contar las inversiones internas de cada mitad y las cruzadas al mezclar
    inv_izq <- MergeSortInversiones(L, izq, mid)
    inv_der <- MergeSortInversiones(L, mid + 1, der)
    inv_cruzadas <- MergeYContar(L, izq, mid, der)

    Retornar inv_izq + inv_der + inv_cruzadas
FinFuncion

Funcion ContarInversiones(L, n)
Inicio
    // Llamada inicial para procesar el vector completo L[1..n]
    Retornar MergeSortInversiones(L, 1, n)
FinFuncion

```

d) Análisis del Coste Asintótico en Tiempo

1. **Ecuación de Recurrencia:** El tiempo de ejecución $T(n)$ para procesar un arreglo de tamaño n se descompone en resolver dos subproblemas de tamaño $n/2$ de manera recursiva más una fase de combinación lineal $O(n)$ correspondiente a la fusión de subarreglos:

$$T(n) = 2T\left(\frac{n}{2}\right) + O(n) = T(n) \in O(n \log n)$$